Graduate Algorithms

Quiz 2

August 17, 2026

Had Arjun agreed silently with Krishna, there would be no Gita. ASK QUESTIONS

Instructions

- Total marks: 20. Time: 40 minutes.
- There are 6 problems; attempt all. Write your answers in the space provided on this sheet. You are advised to spend at most 15 minutes on first four questions.
- Additional sheets are not allowed. Space for rough work is provided on page 2. Throughout, log means $\log_2$.

Name: ______________________________ **Roll Number:** ________________

Question 1: Master Theorem

3 marks

Consider the recurrence $T(n) = a\,T(n/b) + f(n)$, where $a > 0$, $b > 1$ and $f(n)$ is asymptotically positive, and let $n^{\log_b a}$ be the critical exponent.

1. If $f(n) = O\left(n^{\log_b a - \varepsilon}\right)$ for some constant $\varepsilon > 0$, then $T(n) = \Theta\left(n^{\log_b a}\right)$.
2. If $f(n) = \Theta\left(n^{\log_b a} \log^k n\right)$ for some $k \geq 0$, then $T(n) = \Theta\left(n^{\log_b a} \log^{k+1} n\right)$.
3. If $f(n) = \Omega\left(n^{\log_b a + \varepsilon}\right)$ for some $\varepsilon > 0$, and $a\,f(n/b) \leq c\,f(n)$ for some $c < 1$ and all large n, then $T(n) = \Theta(f(n))$.

For each recurrence below, fill in the table: whether *this* Master Theorem applies, which case, and the asymptotic solution. Where it does not apply, write "N/A" in the last two columns, you need not explain why. You need not show intermediate steps when the solution follows directly from the theorem.

Recurrence	**MT applicable? (Y/N)**	**Case (1/2/3)**	**Asymptotic solution**
$T(n) = 6T(n/3) + n^2 \log n$			
$T(n) = 29T(n/29) + n/\log n$			

Question 2: Comparing Growth Rates

4 marks

Each cell below gives a pair of functions f and g. Mark every relation with a tick if it holds for that pair and a cross if it does not; do not leave any entry blank. More than one relation may hold for a single pair. Here O, Ω and Θ abbreviate $f = O(g)$, $f = \Omega(g)$ and $f = \Theta(g)$. Marks will be awarded only if all three entries in a cell are correct. The first row is worked out as examples.

$f(n)$	$g(n)$	O	Ω	Θ	$f(n)$	$g(n)$	O	Ω	Θ
n^{29}	$2004^{29} \cdot n$	$\times$	$\checkmark$	$\times$	29^n	$(29n)!$	$\checkmark$	$\times$	$\times$
$\log^{29} n$	$n^{0.29}$				$n^{\log 29}$	$29^{\log n}$			
$n^{0.29}$	$(0.29)^n$				$\log(\sqrt[29]{n})$	$\sqrt[29]{\log n}$			

Question 3: Ordering by Growth

3 marks

Arrange the following six functions in increasing order of growth, that is, produce an ordering $g_1, g_2, \ldots, g_6$ with $g_{i+1} = \Omega(g_i)$ for each i. If any two of them are Θ of each other, say so explicitly. Write only the ordering, no working is required. Marks are awarded only for a completely correct ordering.

$$n!, \qquad \left(\sqrt{2}\right)^{\log n}, \qquad n^2, \qquad \log(n!), \qquad \log^5 n, \qquad (3/2)^n$$

Question 4: The Four-Item Sorter

2 marks

A piece of hardware performs a single primitive operation: handed any four of the items, it returns those four in sorted order. No other operation on the items is available. Consider algorithms that sort $n \geq 4$ items using this primitive alone. Of the four bounds listed, tick the strongest one that is a valid lower bound on the number of primitive operations used in the worst case.

(a) $\Omega(n)$ (b) $\Omega(n \log n)$ (c) $\Omega(\log n)$ (d) $\Omega(\sqrt{n})$

Space for Rough work

Question 5: Finding the Maximum

5 marks

We work in the comparison model: an algorithm may only compare two elements and learn which is larger, and it is charged one unit per comparison. Let A be an array of $n \geq 2$ distinct elements.

Prove that every deterministic algorithm that finds the maximum of A must make at least $n - 1$ comparisons in the worst case.

Solution:

Question 6: Coarsening Merge Sort

3 marks

Merge sort runs in $\Theta(n \log n)$ worst-case time and insertion sort in $\Theta(n^2)$, but insertion sort has smaller constant factors and so is often faster on short arrays. This suggests *coarsening* the leaves of the recursion: split the input into n/k sublists of length k, sort each one with insertion sort, and then merge them using the usual merging mechanism. Here k is a parameter to be chosen.

You may use, without proof, the fact that the modified algorithm runs in

$$\Theta(nk + n \log(n/k))$$

worst-case time. What is the largest value of k, as a function of n, for which the modified algorithm still matches standard merge sort asymptotically? Justify your answer.

Solution: